



THIẾT KẾ LUẬN LÝ SỐ

CHƯƠNG 4: THIẾT KẾ BỘ XỬ LÝ



Nội dung

❖ Giới thiệu các khái niệm:

- Tập lệnh
- Các loại lệnh
- Các chế độ đánh địa chỉ
- Chu kỳ thực thi lệnh
- Quy trình thiết kế bộ xử lý

❖ Bên cạnh đó trình bày:

- Thiết kế tập lệnh
- Lưu đồ tập lệnh
- Bố trí các khối thành phần
- Lưu đồ ASM
- Kiến trúc bộ xử lý

❖ Thiết kế bộ xử lý:

- Bộ xử lý CISC 16-bit
- Bộ xử lý RISC 32-bit

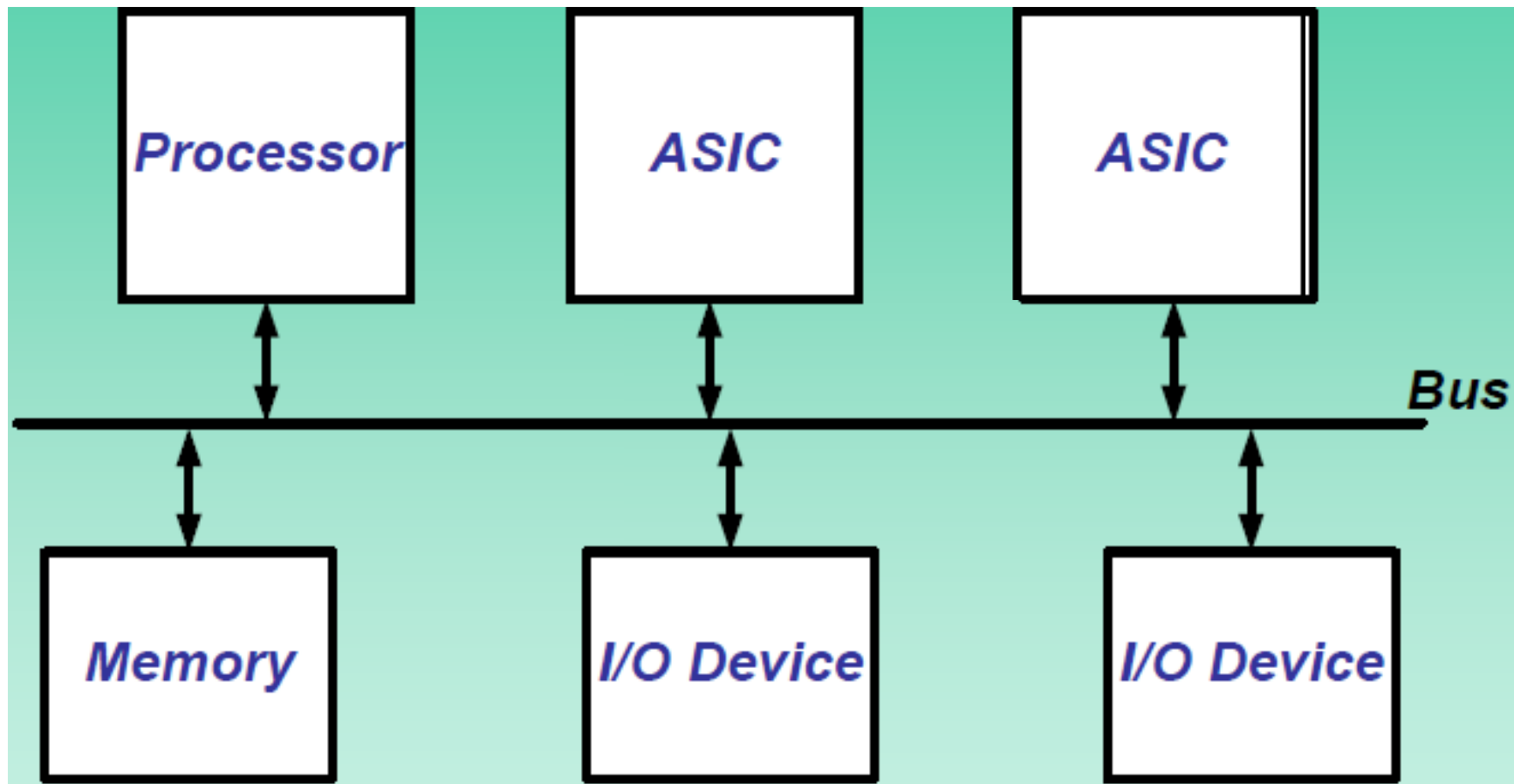


Định nghĩa cơ bản

- **Bộ xử lý** (Processor) kiểm soát toàn bộ hoạt động của hệ thống, giám sát các thiết bị I/O (bàn phím, màn hình, đĩa, băng), đồng bộ hóa dữ liệu giữa hệ thống và các thành phần, và thực hiện hầu hết các tác vụ tính toán.
- **ASIC** (Application Specification Integrated Circuit) là một bộ đồng xử lý thực hiện một hoặc nhiều tác vụ cụ thể nhanh hơn nhiều so với chính bộ xử lý đó. Vì lý do này, các ASIC được gọi như các máy gia tốc, bởi bộ xử lý sẽ chuyển các tác vụ tính toán chuyên sâu cho chúng.



Kiến trúc cơ bản một máy tính





Tập lệnh

& Một số khái niệm



Tập lệnh

- **Tập lệnh** (Instruction Set- IS) là một tập hợp các lệnh và định dạng lệnh, sẽ được giải thích bởi Khối điều khiển và được thực thi bởi Datapath của bộ xử lý.
- Một lệnh là một chuỗi các bit được gom nhóm vào các trường thông tin khác nhau, ví dụ như:
 - Mã tác vụ (op-code)
 - Kiểu lệnh
 - Trường địa chỉ
- Một lệnh cơ bản, ví dụ $a = b + c$, trong đó a, b, c được lưu ở bộ nhớ tại các địa chỉ A, B, và C sẽ được biểu diễn bằng hợp ngữ (assembly language) như sau:
 - **Add A, B, C** (lệnh hợp ngữ) $(\text{Mem}[A] \leftarrow \text{Mem}[B] + \text{Mem}[C])$ (giải thích toán học)



Các lệnh phổ biến: Kiểu lệnh

- **Lệnh trên thanh ghi (số học, luận lý, dịch)**
 - **Add** RA, RB, RC ($RF[A] \leftarrow RF[B] + RF[C]$)
- **Lệnh di chuyển dữ liệu (load và store)**
 - **Load** R2, A ($RF[2] \leftarrow MEM[A]$)
 - **Store** A, R2 ($MEM[A] \leftarrow RF[2]$)
- **Lệnh điều khiển (lệnh rẽ nhánh)**
 - **Beq** R2, R3, A if $R2=R3$ then $PC \leftarrow MEM[A]$
 if $R2 \neq R3$ then $PC \leftarrow PC+1$



Các lệnh phổ biến: Kiểu lệnh

➤ Lệnh thể hiện chế độ địa chỉ

Lind R2, A ($RF[2] \leftarrow Mem[Mem[A]]$)

➤ Lệnh chứa hằng số

Add R2, R3, 1 ($RF[2] \leftarrow RF[3] + 1$)



Số trường địa chỉ vs. Hiệu suất

- Tính biểu thức toán học: $C = (a+b)(a-b)$
- Chương trình với lệnh 3 địa chỉ:
 1. **Add** X, A, B ($\text{Mem}[X] \leftarrow \text{Mem}[A] + \text{Mem}[B]$)
 2. **Sub** C, A, B ($\text{Mem}[C] \leftarrow \text{Mem}[A] - \text{Mem}[B]$)
 3. **Mul** C, X, C ($\text{Mem}[C] \leftarrow \text{Mem}[X] \times \text{Mem}[C]$)
- Kích cỡ chương trình: 3 lệnh
- Hiệu suất: **9** lần truy cập bộ nhớ để nạp 3 lệnh, **9** lần truy cập bộ nhớ để lấy toán hạng và lưu kết quả
- ✓ Giả sử kích cỡ bộ nhớ là 16 đến 256 triệu từ nhớ $\rightarrow$ mỗi địa chỉ chiếm 24 đến 28 bits $\rightarrow$ 3 địa chỉ trong mỗi lệnh cần 80 đến 90 bits (bao gồm cả trường tác vụ)
- ✓ Giả sử bộ vi xử lý 32-bit và bộ nhớ là 32-bit words $\rightarrow$ mỗi lệnh sẽ chiếm 3 words trong bộ nhớ



Số trường địa chỉ vs. Hiệu suất

- Tính biểu thức toán học sau: $C = (a+b)(a-b)$
- Chương trình hợp ngữ với các **lệnh hai địa chỉ** (dữ liệu tạm trong bộ nhớ):
 1. **Move** X, A ($\text{Mem}[X] \leftarrow \text{Mem}[A]$)
 2. **Add** X, B ($\text{Mem}[X] \leftarrow \text{Mem}[X] + \text{Mem}[B]$)
 3. **Move** C, A ($\text{Mem}[C] \leftarrow \text{Mem}[A]$)
 4. **Sub** C, B ($\text{Mem}[C] \leftarrow \text{Mem}[C] - \text{Mem}[B]$)
 5. **Mul** C, X ($\text{Mem}[C] \leftarrow \text{Mem}[C] \times \text{Mem}[X]$)
- Kích cỡ chương trình: 5 lệnh
- Hiệu suất: **10** lần truy cập bộ nhớ cho lệnh, **13** lần truy cập bộ nhớ cho dữ liệu



Số trường địa chỉ vs. Hiệu suất

- Tính biểu thức toán học sau: $C = (a+b)(a-b)$
- Chương trình hợp ngữ với các **lệnh một địa chỉ** (dữ liệu tạm trong thanh ghi tích lũy **ACC**):
 1. **Load A** ($ACC \leftarrow Mem[A]$)
 2. **Add B** ($ACC \leftarrow ACC + Mem[B]$)
 3. **Store X** ($Mem[X] \leftarrow ACC$)
 4. **Load A** ($ACC \leftarrow Mem[A]$)
 5. **Sub B** ($ACC \leftarrow ACC - Mem[B]$)
 6. **Mul X** ($ACC \leftarrow ACC \times Mem[X]$)
 7. **Store C** ($Mem[C] \leftarrow ACC$)
- Kích cỡ chương trình: 7 lệnh
- Hiệu suất: 7 lần truy cập bộ nhớ cho lệnh, 7 lần truy cập bộ nhớ cho dữ liệu



Số trường địa chỉ vs. Hiệu suất

- Tính biểu thức toán học sau: $C = (a+b)(a-b)$
- Chương trình hợp ngữ với các **lệnh hai địa chỉ** (dữ liệu tạm lưu trong **tập thanh ghi**):

1. . Load R1, A	($RF[1] \leftarrow Mem[A]$)
2. Load R2, B	($RF[2] \leftarrow Mem[B]$)
3. Move R3, R2	($RF[3] \leftarrow RF[2]$)
4. Add R1, R2	($RF[1] \leftarrow RF[1] + RF[2]$)
5. Sub R3, R2	($RF[3] \leftarrow RF[3] - RF[2]$)
6. Mul R1, R3	($RF[1] \leftarrow RF[1] \times RF[3]$)
7. Store C, R1	($Mem[C] \leftarrow RF[1]$)

- Kích cỡ chương trình: 7 lệnh
- Hiệu suất: **7** lần truy cập bộ nhớ cho lệnh, **3** lần truy cập bộ nhớ (2 Load và 1 Store) cho dữ liệu



Số trường địa chỉ vs. Hiệu suất

- Tính biểu thức toán học sau: $C = (a+b)(a-b)$
- Chương trình hợp ngữ với các **lệnh 3 địa chỉ từ thanh ghi** và **lệnh 2 địa chỉ từ bộ nhớ** (dữ liệu tạm lưu trong **tập thanh ghi**):

1. Load R1, A	($RF[1] \leftarrow Mem[A]$)
2. Load R2, B	($RF[2] \leftarrow Mem[B]$)
3. Add R3, R1, R2	($RF[3] \leftarrow RF[1] + RF[2]$)
4. Sub R4, R1, R2	($RF[4] \leftarrow RF[1] - RF[2]$)
5. Mul R5, R4, R3	($RF[5] \leftarrow RF[3] \times RF[4]$)
6. Store C, R5	($Mem[C] \leftarrow RF[5]$)

- Kích cỡ chương trình: 6 lệnh
- Hiệu suất: **6** lần truy cập bộ nhớ cho lệnh, **3** lần truy cập bộ nhớ cho dữ liệu



Chế độ đánh địa chỉ

Ẩn (Implied)

(set, reset ACC/status registers)

Op code

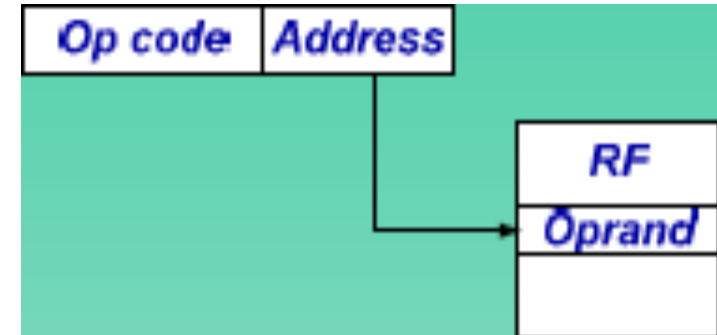
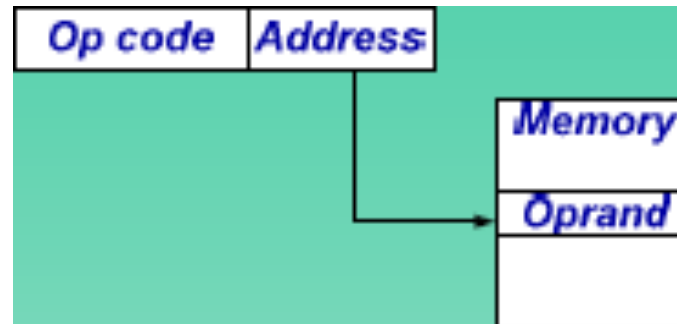
Số tức thời (Immediate)

(incre/decre loop,
array indices, coefficient)

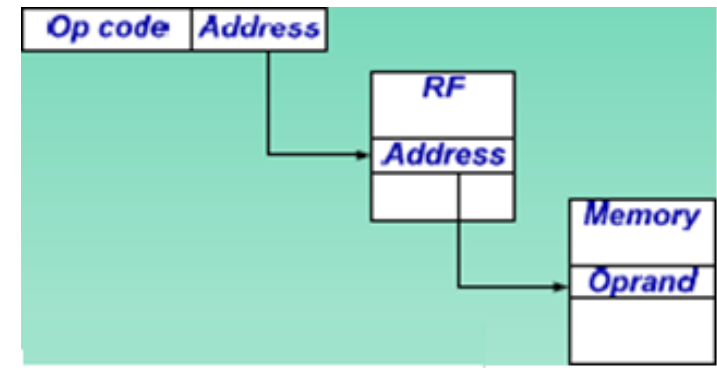
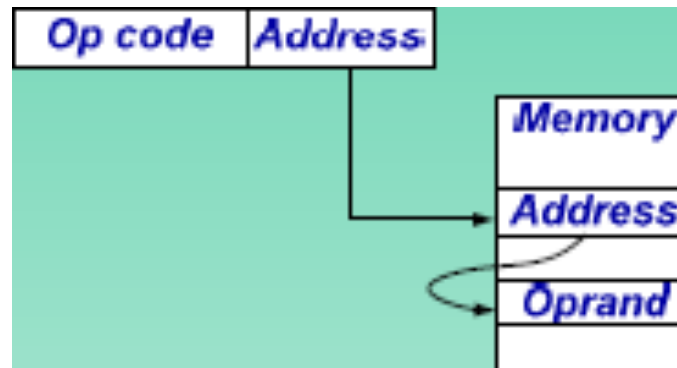
Op code

Oprand

Trực tiếp (Direct)



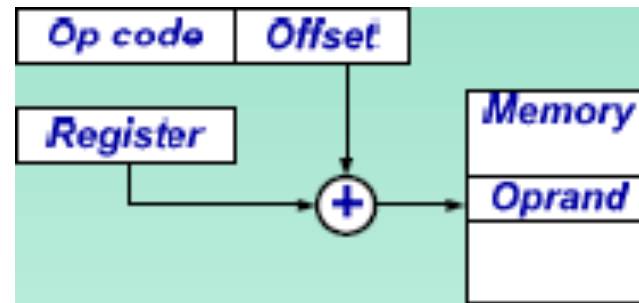
Gián tiếp (Indirect)



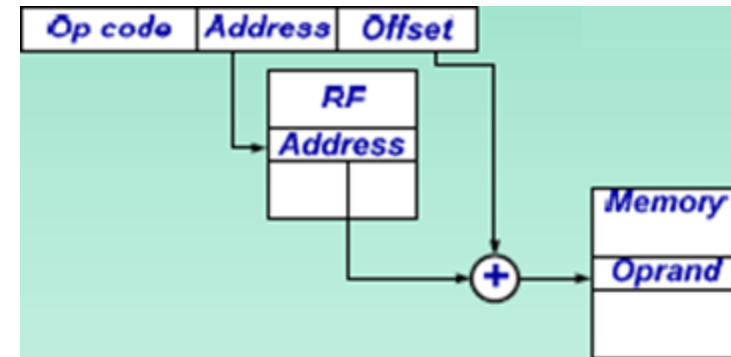


Chế độ đánh địa chỉ

Tương đối (Relative)



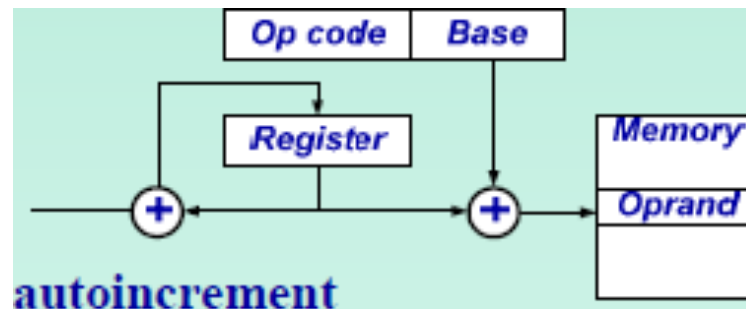
Lệnh rẽ nhánh



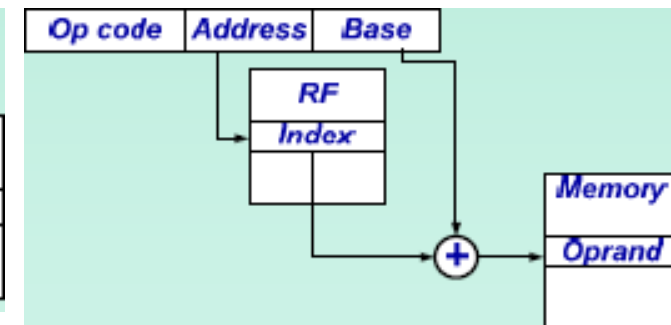
Tra bảng

Chỉ mục (Indexed)

“1”



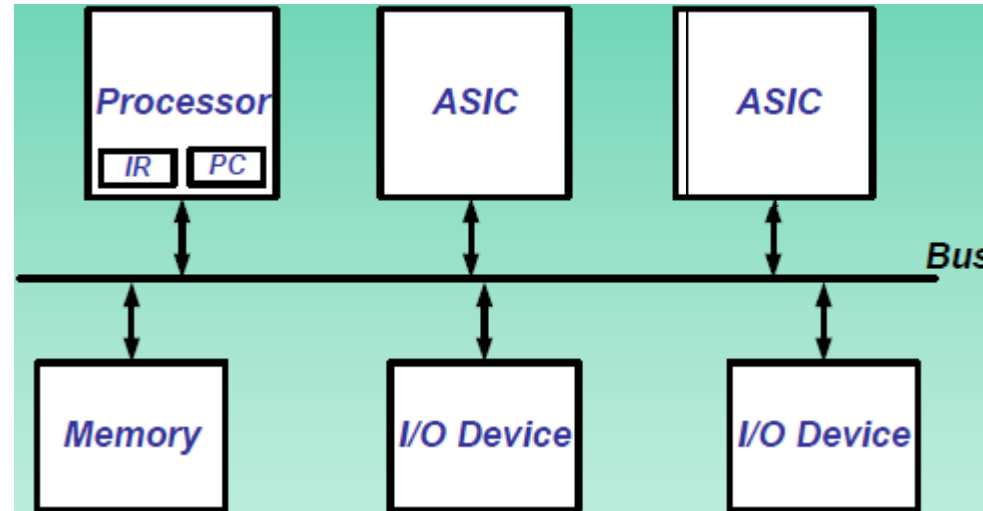
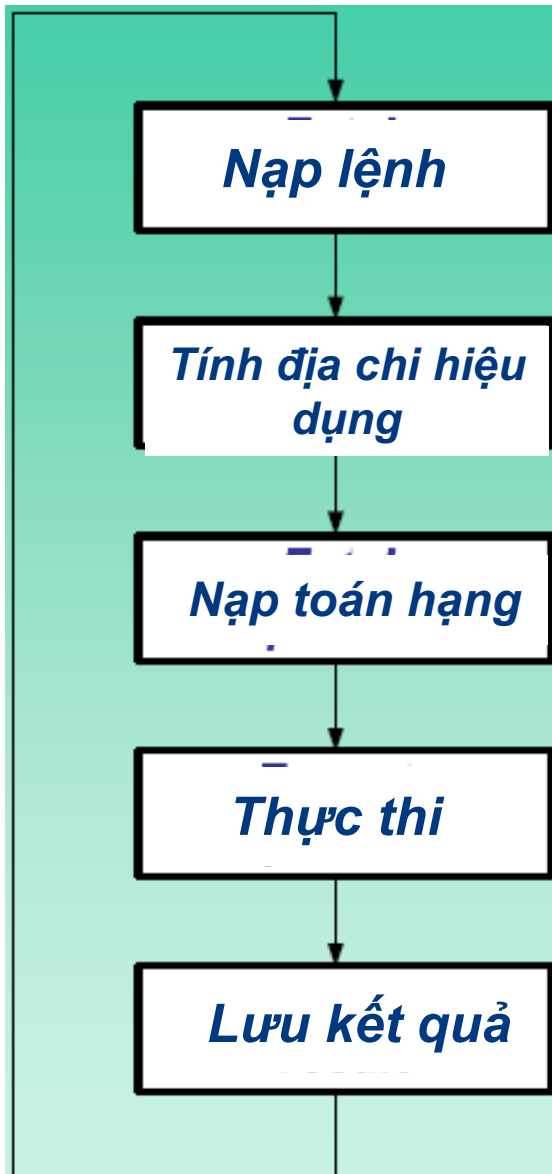
Truy cập stack hoặc queue



Truy cập mảng

Tìm hiểu sự khác nhau giữa chế độ đánh địa chỉ “Relative” và “Indexed” ?

Chu kỳ thực thi lệnh



IR = Instruction Register

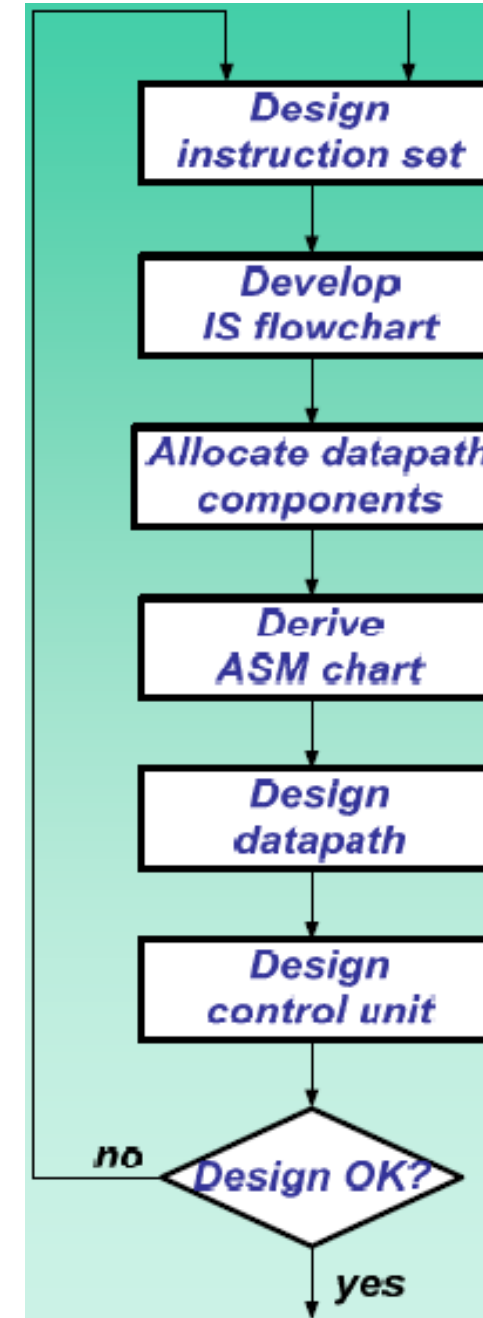
PC = Program Counter

Memory stores all instructions and data

Quy trình thiết kế bộ xử lý

Cân đối giữa **tính hiệu quả và kích cỡ** của chương trình với **chi phí và hiệu suất** của bộ xử lý

- ❖ **Tập lệnh (IS):** chỉ ra hoạt động của mỗi lệnh.
- ❖ **Lưu đồ tập lệnh (IS flowchart) :** mô tả chính xác các thao tác được thực thi trong mỗi lệnh.
- ❖ **Sơ đồ ASM:** mô tả thực thi lệnh qua các chu kỳ clock
- ❖ **Thiết kế datapath:** hoàn thành sơ đồ kết nối.



Có thể
lặp lại
nhiều
lần



Thiết kế tập lệnh

- Xem xét tính hiệu quả của chương trình vs. kích thước chương trình
- Chi phí bộ xử lý vs. hiệu suất bộ xử lý
- Cân đối giữa **kích thước chương trình** với **kích thước bộ xử lý**
- **CISC** vs. **RISC**



Thiết kế tập lệnh

- **Tập lệnh phức tạp (Complex Instruction-set - CISC)**
 - lệnh mạnh mẽ -> chương trình ngắn hơn
 - lệnh mạnh mẽ -> datapath, control unit phức tạp
 - lệnh phức tạp hơn -> nhiều chu kỳ clock
 - datapath, control unit phức tạp -> chu kỳ clock dài hơn
 - lệnh phức tạp -> pipeline kém
- **Tập lệnh đơn giản (Reduced Instruction-set - RISC)**
 - lệnh đơn giản-> chương trình dài hơn
 - lệnh đơn giản- -> datapath, control unit đơn giản
 - lệnh đơn giản- -> một chu kỳ clock
 - datapath đơn giản -> chu kỳ clock ngắn hơn
 - lệnh đơn giản -> pipeline tốt



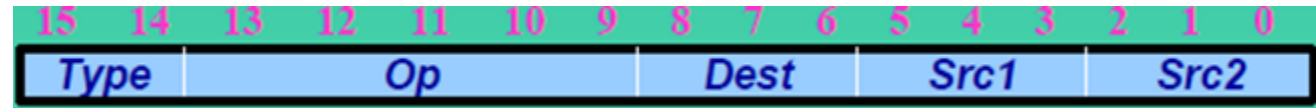
Thiết kế bộ xử lý CISC & RISC



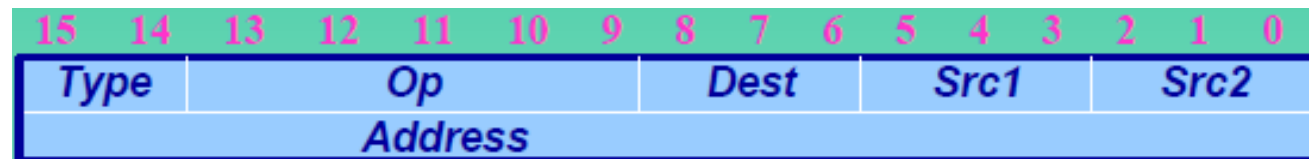
Tập lệnh CISC cho bộ xử 16-bit

Mỗi lệnh kích thước $\leftarrow$ 1 từ 16-bit (không truy cập bộ nhớ) (2-bit Type field, 5-bit Op field,...)
 $\leftarrow$ 2 từ 16-bit (truy cập bộ nhớ trong từ thứ 2) $\rightarrow$ truy cập bộ nhớ 64K

a) Lệnh thanh ghi
toán học, luận lý, di
chuyển, và dịch



Name	Action
<i>Op Dest, Src1, Src2</i>	$RF(Dest) \leftarrow RF(Src1) \text{ Op } RF(Src2)$



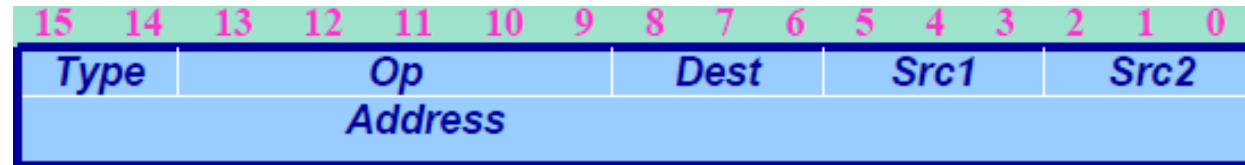
b) Lệnh bộ nhớ
load và store

Name	Name Action
<i>L imm Dest</i>	$RF[Dest] \leftarrow Address$
<i>L dir Dest</i>	$RF[Dest] \leftarrow Mem[Address]$
<i>L rel Dest, Src2</i>	$RF[Dest] \leftarrow Mem[RF[Src2]+Address]$
<i>L in Dest</i>	$RF[Dest] \leftarrow Mem[Mem[Address]]$
<i>S dir Src1</i>	$Mem[Address] \leftarrow RF[Src1]$
<i>S rel Src1, Src2</i>	$Mem[RF[Src2]+Address] \leftarrow RF[Src1]$
<i>S in Src1</i>	$Mem[Mem[Address]] \leftarrow RF[Src1]$



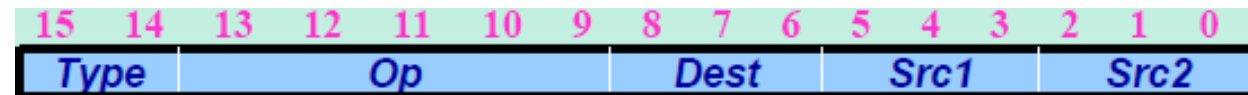
Tập lệnh CISC cho bộ xử 16-bit

c) **Lệnh điều khiển**
jump, branch, call và
return



Name	Action
Jump Address	$PC \leftarrow \text{Address}$
Brel Address	$PC \leftarrow PC+1$ if Status[rel] = 0
	$PC \leftarrow \text{Address}$ if Status[rel] = 1
Call Address, Src1	$\text{Mem}[\text{RF}[\text{Src1}]] \leftarrow PC+1; PC \leftarrow \text{Address};$ $\text{RF}[\text{Src1}] \leftarrow \text{RF}[\text{Src1}] + 1$
Return	$\text{RF}[\text{Src1}] \leftarrow \text{RF}[\text{Src1}] - 1;$ $PC \leftarrow \text{Mem}[\text{RF}[\text{Src1}]] ;$

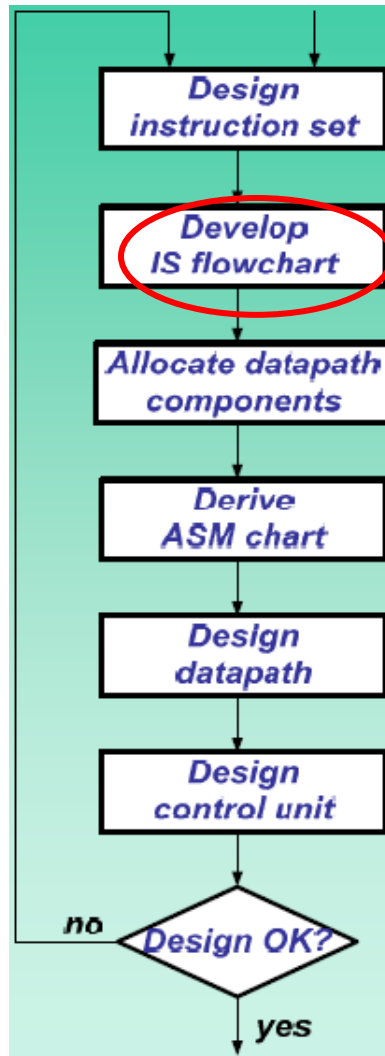
d) **Các lệnh khác**
no-op, clear, status, set
and reset



Name	Action	six relations
No-op	Do nothing	
Clear Dest	$\text{RF} [\text{Dest}] \leftarrow 0$	
Lstat Src1, Src2	$\text{Status} \leftarrow \text{R}[\text{Src1}] \text{ } \text{>=<} \text{ RF}[\text{Src2}]$	
Sstat Dest	$\text{Status} [\text{Dest}] \leftarrow 1$	
Rstat Dest	$\text{Status} [\text{Dest}] \leftarrow 0$	



Thiết kế CISC: Lưu đồ tập lệnh



Quy trình thiết kế
bộ xử lý

Lưu đồ tập lệnh (IS flowchart)

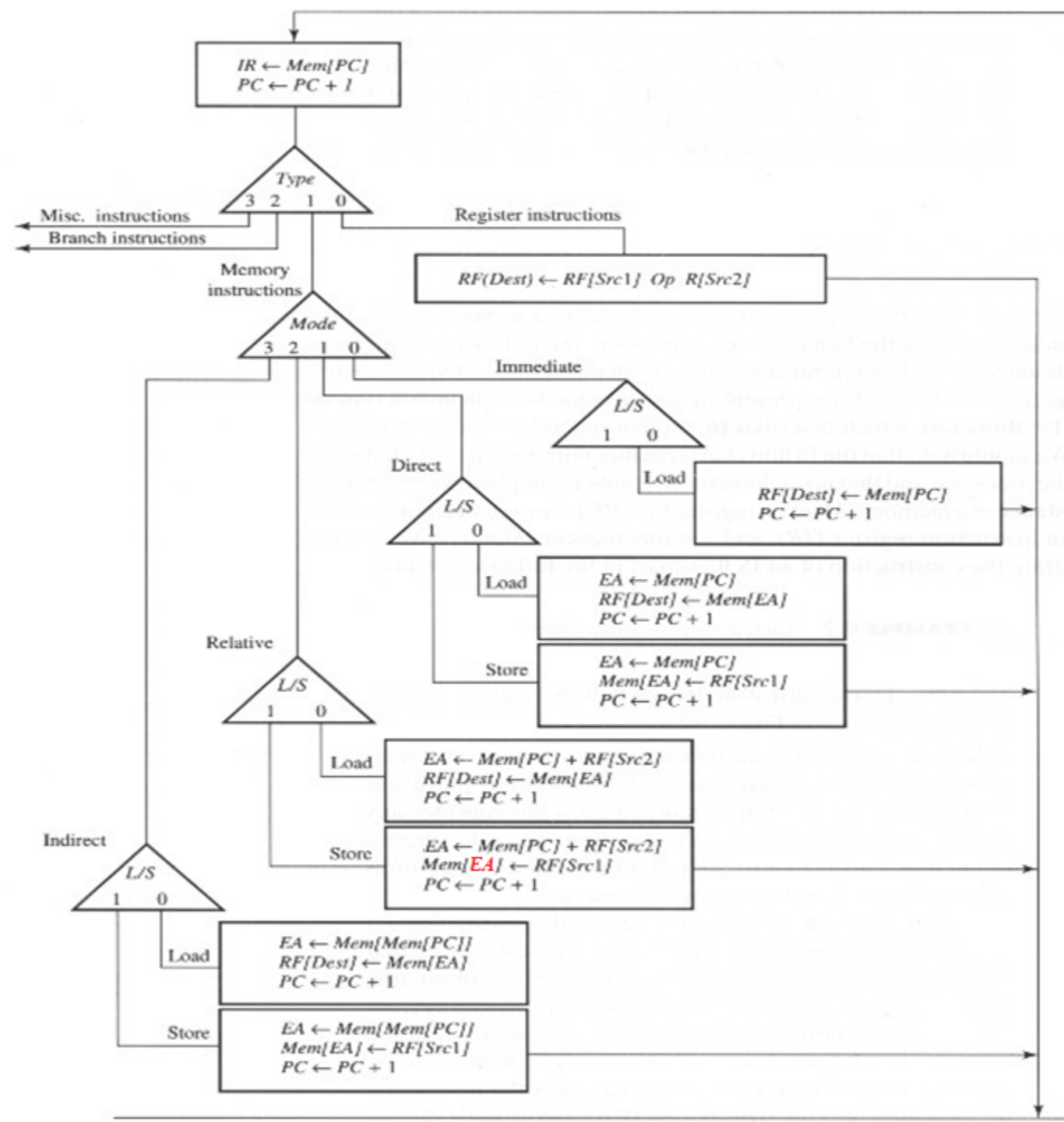
- Không mô tả chi tiết thiết kế
- Không đưa ra sơ đồ datapath cụ thể
- Không xét đến các ràng buộc thời gian và độ dài chu kỳ clock

Mục đích:

- Chỉ đưa ra thứ tự các thao tác thực thi trong mỗi lệnh.

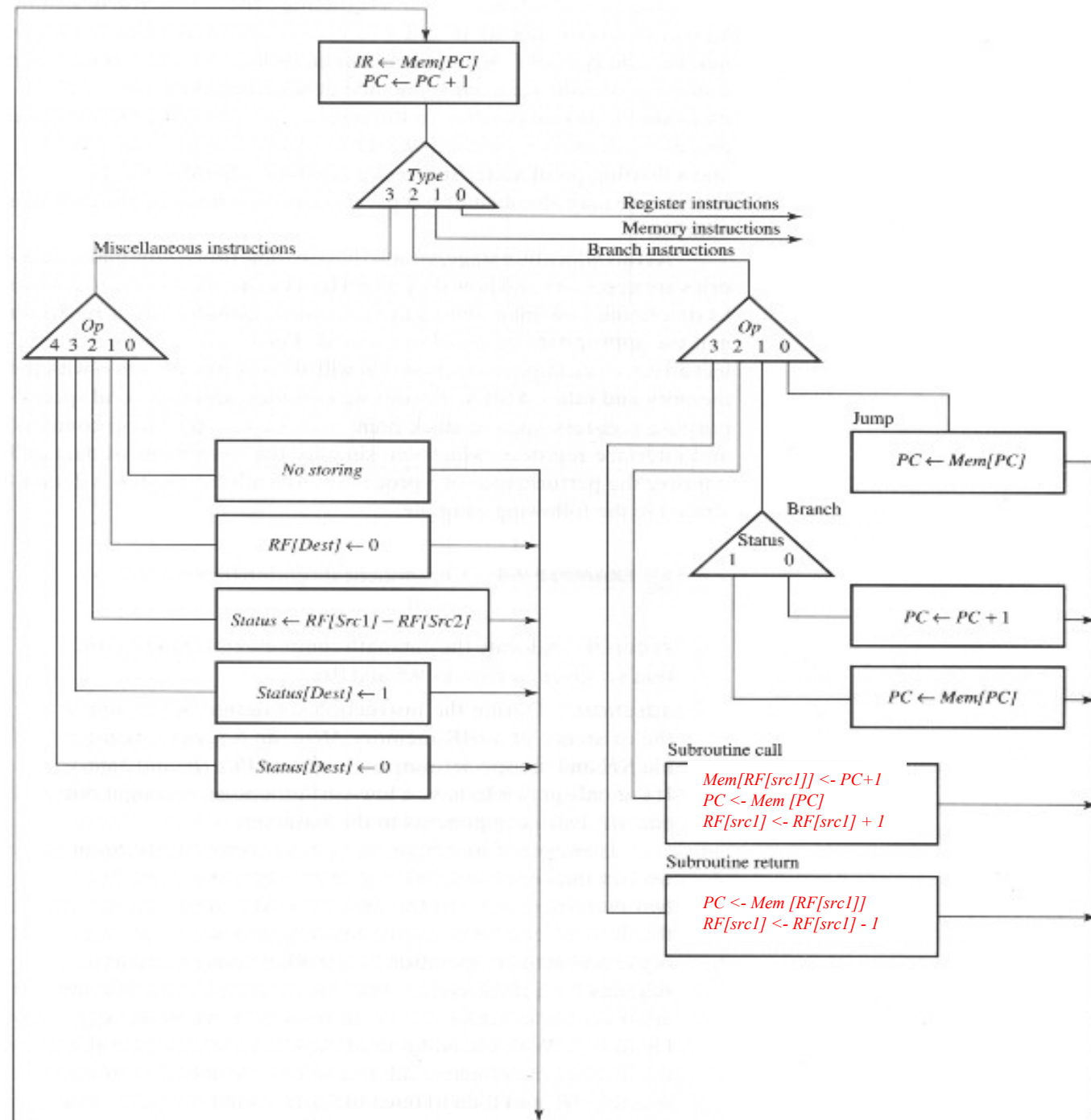


Thiết kế CISC: Lưu đồ tập lệnh



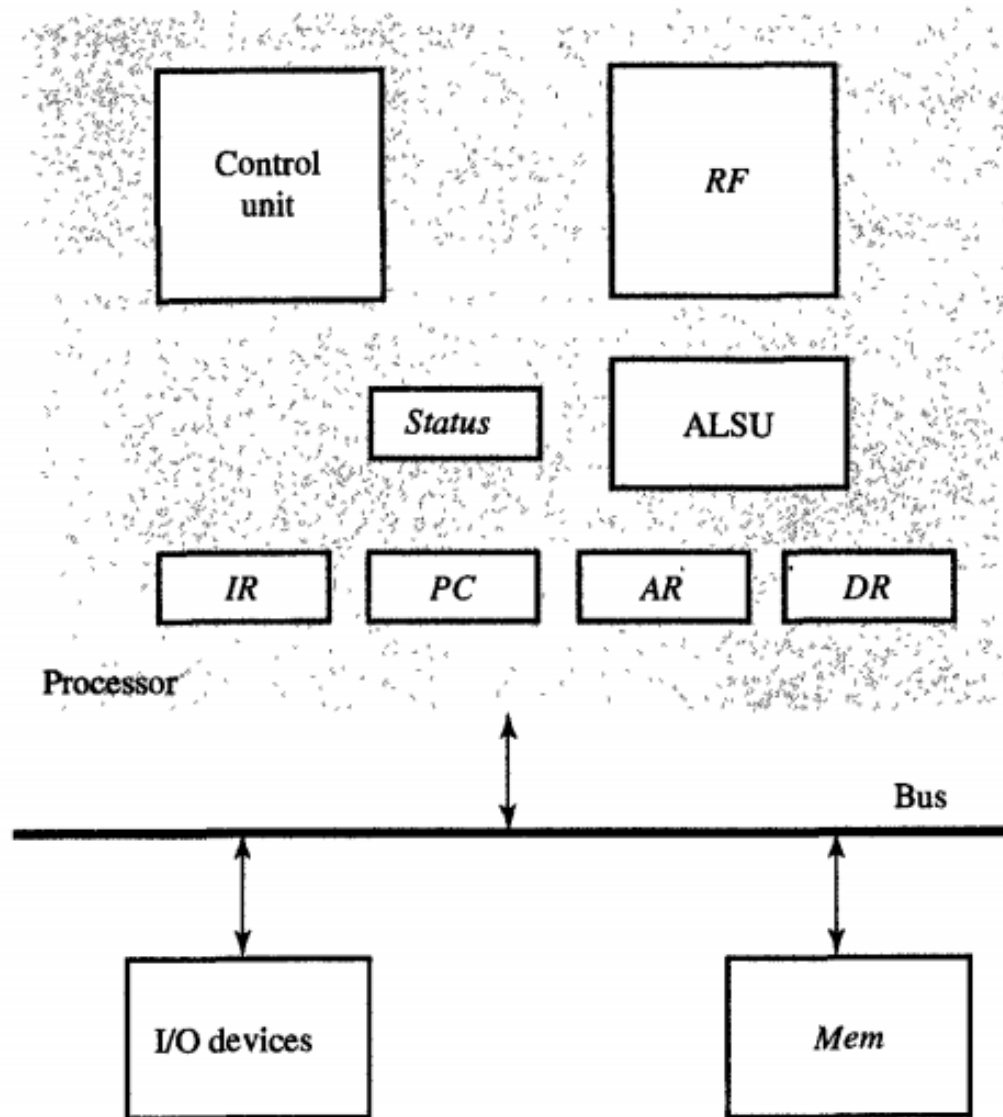


Thiết kế CISC: Lưu đồ tập lệnh





Đặt các thành phần chức năng cho bộ xử lý 16-bit



Các thành phần: 64K x 16 Memory

8 x 16 Register file

ALSU

Instruction register (IR)

Program counter (PC)

Address register (AR)

Data register (DR)

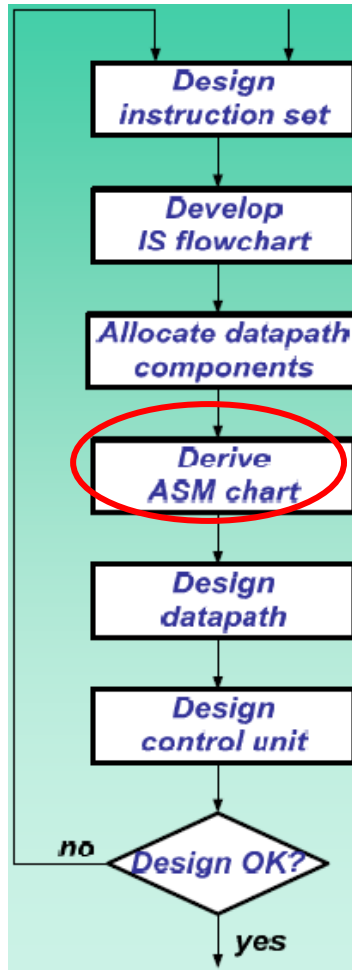
Status register (Status)

Control unit

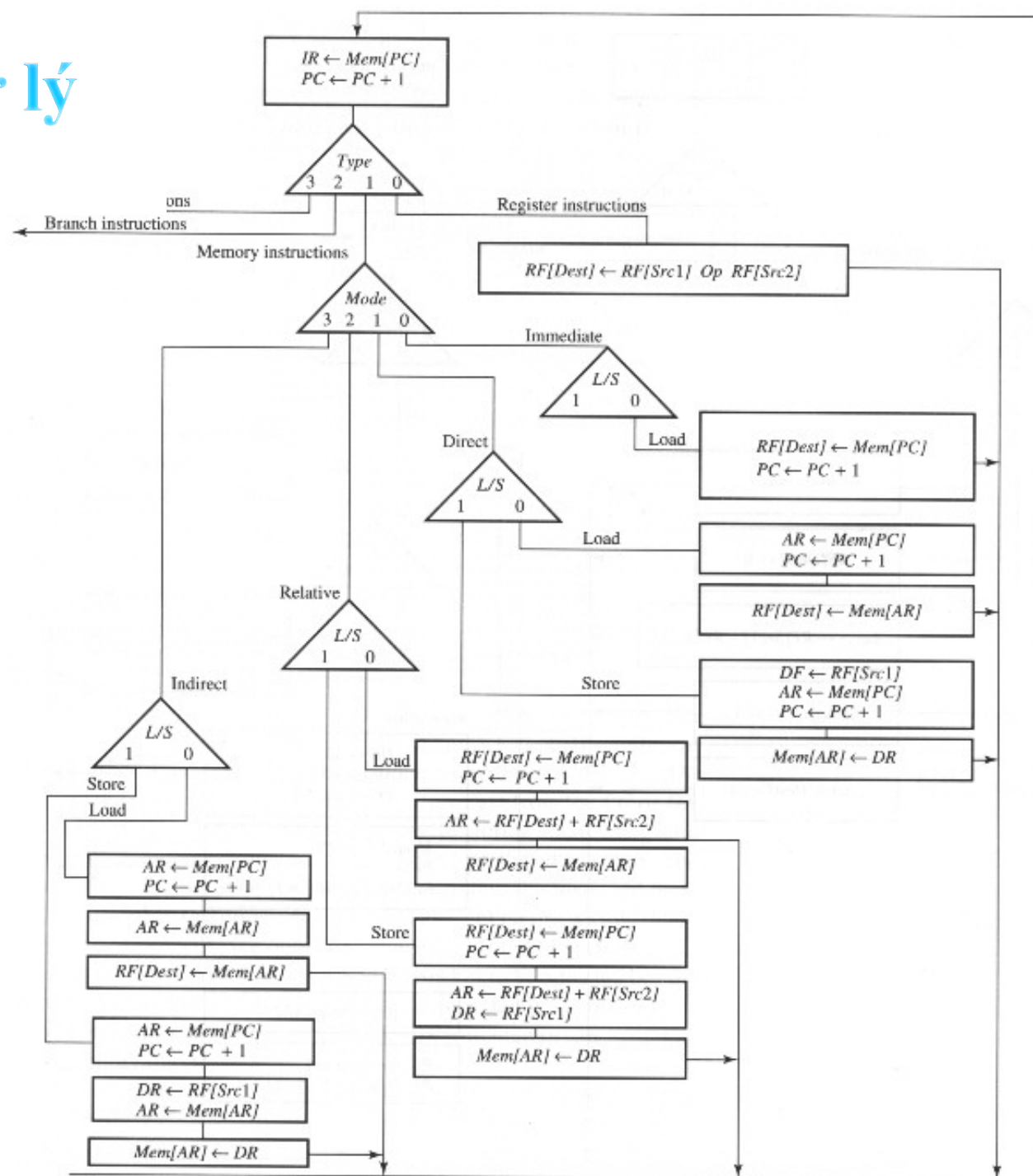
AR, DR: được dùng để rút ngắn chu kỳ clock và cải thiện hiệu suất bộ xử lý



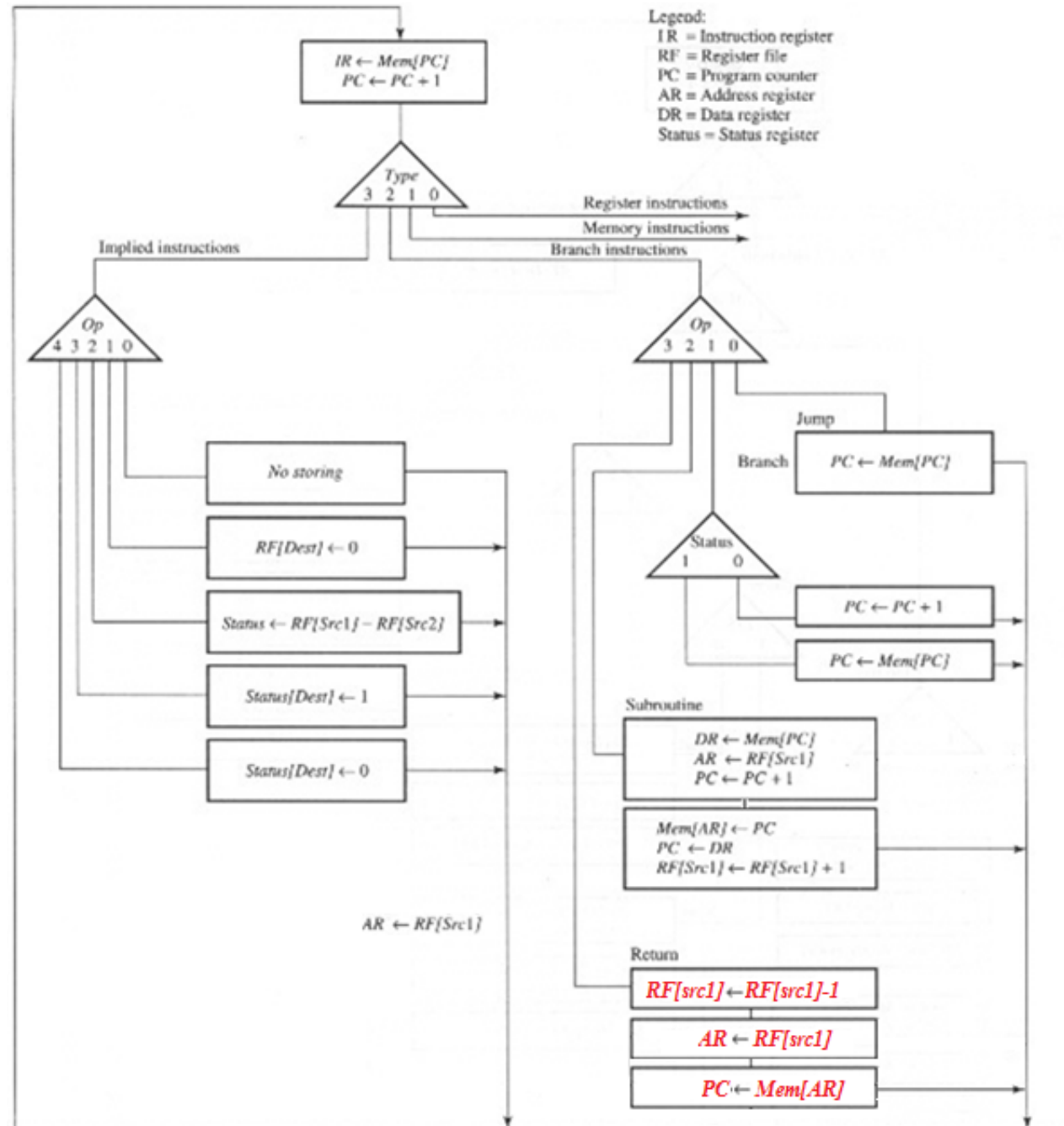
Sơ đồ ASM cho bộ xử lý (Lưu đồ IS định thời)



Quy trình thiết kế
bộ xử lý

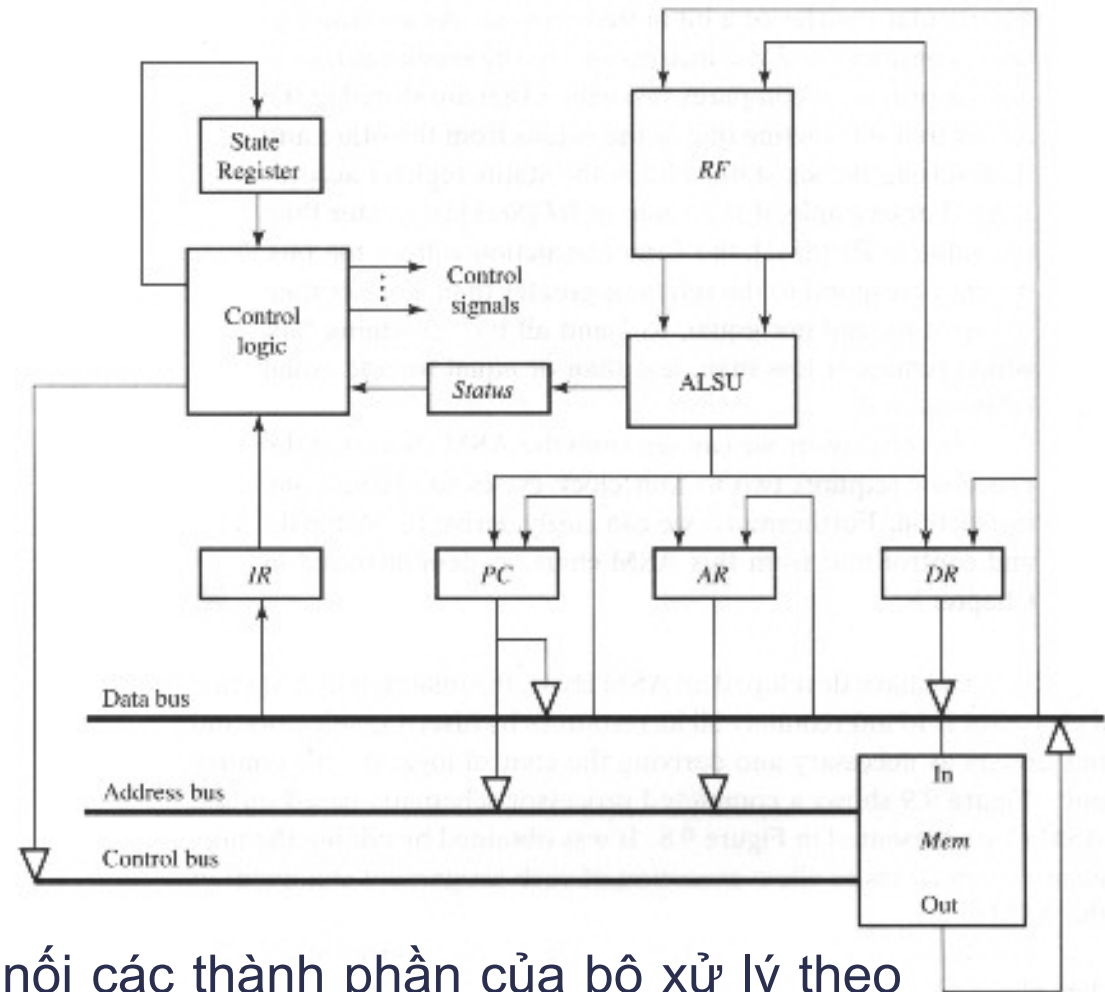
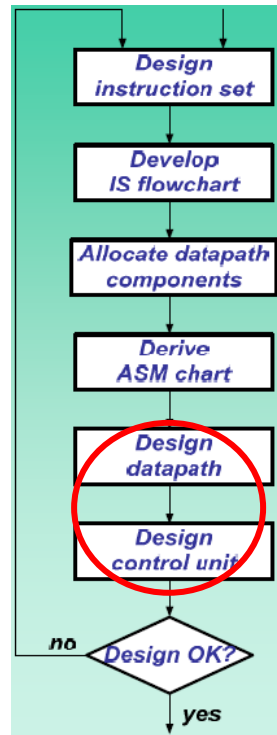


Sơ đồ ASM cho bộ xử lý (Lưu đồ IS định thời)





Sơ đồ kết nối bộ xử lý

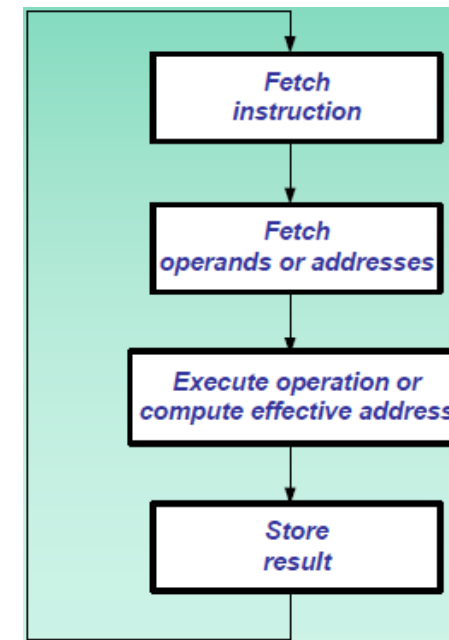


- Sơ đồ có được bằng cách kết nối các thành phần của bộ xử lý theo biểu đồ ASM, nghĩa là, thêm đường kết nối khi có dữ liệu hoặc lệnh được chuyển từ thành phần này sang thành phần khác.
- Các thành phần có nhiều kết nối tại cùng một cổng đầu vào yêu cầu bộ chọn cho cổng đầu vào cụ thể đó.



Thiết kế bộ xử lý RISC

- Register and miscellaneous instructions do not need address fetch and effective address computation
- Memory and control instructions do not need operand fetch and operation execution
- Share operand fetch and address fetch
- Share operand execution and effective address computation
- Thus, instruction cycle reduced to 4 steps



Chu trình thực thi lệnh

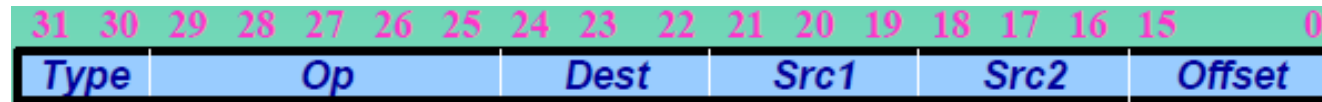
Clock cycle	0	1	2	3	4	5
Fetch instruction	I_1	I_2	I_3	I_4	I_5	I_6
Fetch operands or addresses		I_1	I_2	I_3	I_4	I_5
Execute operation or compute effective address			I_1	I_2	I_3	I_4
Store result				I_1	I_2	I_3

Hiện thực pipeline



Tập lệnh đơn giản (RISC) cho bộ xử lý 32-bit

a) **Lệnh thanh ghi**
toán học, luận lý, di chuyển, và dịch



Name	Action
Op Dest, Src1, Src2	$RF(Dest) \leftarrow RF[Src1] \text{ Op } RF[Src2]$
Op Dest, Src1, Src2	$RF(Dest) \leftarrow RF[Src1] \text{ Op Constant}$
Move Dest, Src1	$RF(Dest) \leftarrow RF[Src1]$
Shift Dest, Src1, Constant	$RF(Dest) \leftarrow RF[Src1] \text{ shift Constant}$

b) **Lệnh bộ nhớ**
load và store



2 chế độ địa chỉ:

- Số tức thời
- Tương đối

Name	Action
L immU Dest	$RF[Dest(31...16)] \leftarrow Offset$
L immL Dest	$RF[Dest(15...0)] \leftarrow Offset$
L rel Dest, Src2, Offset	$RF[Dest] \leftarrow Mem[RF[Src2] + Offset]$
S rel Src1, Src2, Offset	$Mem[RF[Src2] + Offset] \leftarrow RF[Src1]$



Tập lệnh đơn giản (RISC) cho bộ xử lý 32-bit

c) Lệnh điều khiển jump và branch



	<u>Name</u>	<u>Action</u>
Beq (=)	<i>Jump Offset</i>	$PC \leftarrow PC + offset$
Bgre (>)		
Bgoeq ($\geq$)	<i>Jump Src2, Offset</i>	$PC \leftarrow RF[Src2] + offset$
Bless (<)		
Bloeq ($\leq$)	<i>Brel Src1, Src2, Offset</i>	$PC \leftarrow PC+1$ if $RF[Src1]$ not rel $RF[Src2]$ $PC \leftarrow PC+Offset$ if $RF[Src1]$ rel $RF[Src2]$
Bneq ($\neq$)		

d) Các lệnh khác no-op, clear, set và reset

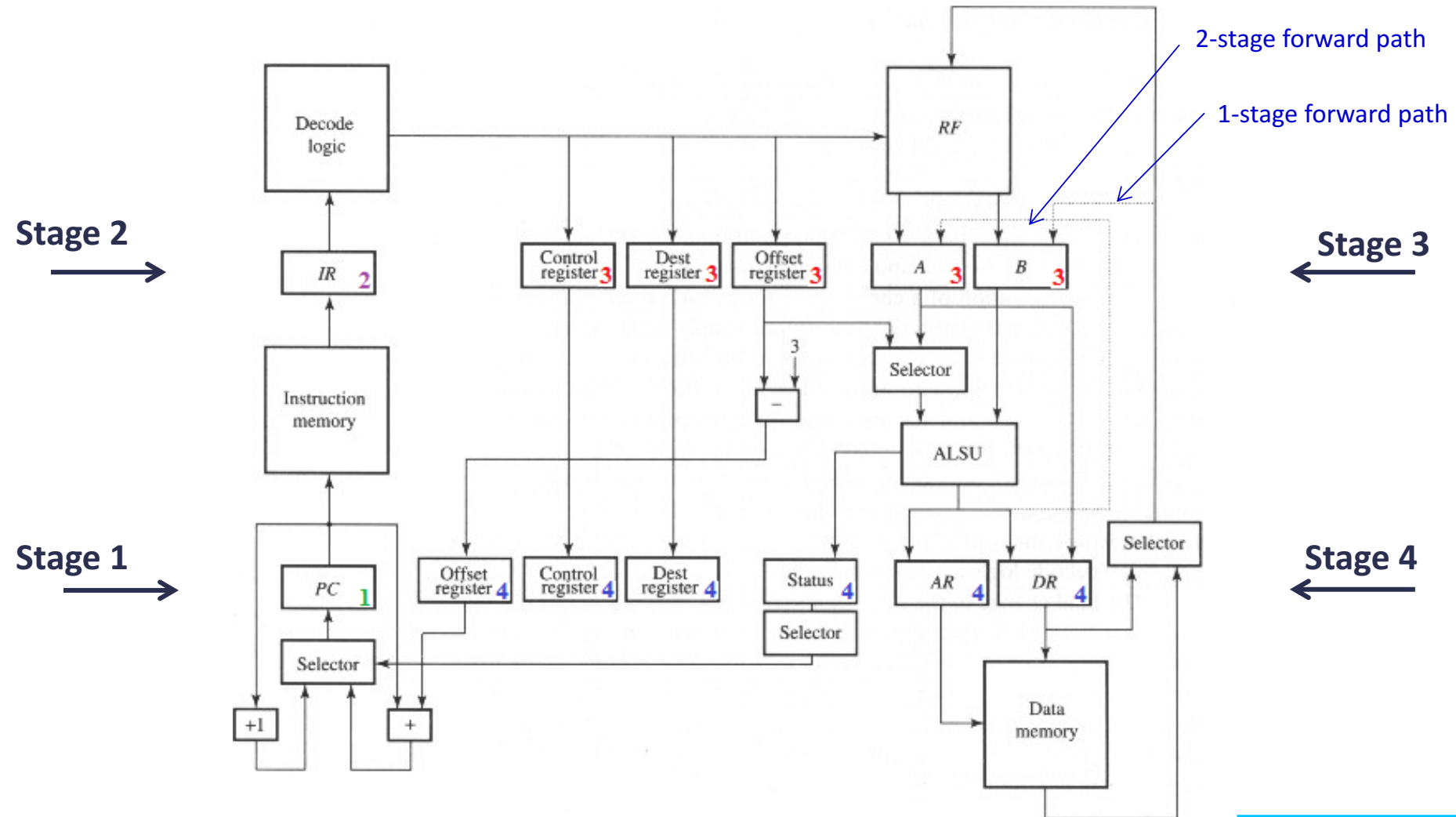


<u>Name</u>	<u>Action</u>
No-op	Do nothing
<i>Clear Dest</i>	$RF [Dest] \leftarrow 0$
<i>Sstat Dest</i>	$status [Dest] \leftarrow 1$
<i>Rstat Dest</i>	$status [Dest] \leftarrow 0$



Sơ đồ khối RISC

- Pipeline 4 tầng
- Tách riêng bộ nhớ lệnh và bộ nhớ dữ liệu
- Thêm vào thanh ghi điều khiển tại mỗi tầng pipeline
- Pipeline stalling (flushing) for control instructions





Hoạt động bộ xử lý RISC

$x = a + b$

$y = b - c$

$z = c + d$

Chương trình nguồn

Song song:

$n+3$ chu kỳ clock để thực thi n lệnh

T tuần tự:

$4n$ clock cho n lệnh (không pipeline)

Address	Instruction
100	Load a, base, off
101	Load b, base, off
102	Load c, base, off
103	Load d, base, off
104	Add x, a, b
105	Sub y, b, c
106	Add z, c, d
107	Store x, base, off
108	Store y, base, off
109	Store z, base, off
110	—

Chương trình hợp ngữ

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Read PC	100	101	102	103	104	105	106	107	108	109	110			
Write IR	Load	Load	Load	Load	Add	Sub	Add	Store	Store	Store				
Write A						a	b	c	x	y	z			
Write B		base	base	base	base	b	c	d	base	base	base			
Write AR			base +off	base +off	base +off	base +off				base +off	base +off	base +off		
Write DR							a+b	b-c	c+d	x	y	z		
Write RF				a	b	c	d	x	y	z				
Write Mem											x	y	z	
Write PC	101	102	103	104	105	106	107	108	109	110	111	112	113	114

Giản đồ thời gian



Hoạt động RISC khi có sự phụ thuộc dữ liệu

$\text{sum} = a + b$

$\text{total} = \text{sum} + c$

Chương trình nguồn

Sự phụ thuộc dữ liệu:

5 (hoặc 45%) của 11 lệnh là lệnh No-op

→ làm giảm đáng kể hiệu suất của bộ xử lý pipeline

Address	Instruction
100	Load a, base, offa
101	Load b, base, offb
102	Load c, base, offc
103	No-op
104	Add sum, a, b
105	No-op
106	No-op
107	Add total, c, sum
108	No-op
109	No-op
110	Store total, base, offt

Chương trình hợp ngữ

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Read PC	100	101	102	103	104	105	106	107	108	109	110			
Write IR	Load	Load	Load	No-op	Add	No-op	No-op	Add	No-op	No-op	Store			
Write A						a			Sum			total		
Write B		base	base	base		b			c			base		
Write AR			base +offa	base +offb	base +offc								base +offt	
Write DR							a+b			sum+c			total	
Write RF				a	b	c		sum			total			
Write Mem														total
Write PC	101	102	103	104	105	106	107	108	109	110	111	112	113	114

Giản đồ thời gian



Hoạt động RISC dùng kỹ thuật nhìn trước

sum = a + b

total = sum + c

Chương trình nguồn

Address	Instruction
100	Load a, base, off
101	Load b, base, off
102	Load c, base, off
103	Add sum, a, b
104	Add total, c, sum
105	Store total, base, off

Chương trình hợp ngữ

Clock cycle	0	1	2	3	4	5	6	7	8
Read PC	100	101	102	103	104	105			
Write IR	Load	Load	Load	Add	Add	Store			
Write A					a	$F2[a+F1[B]]$	$F2[(a+F1[B]) + F1[C]]$		
Write B		base	base	base	$F1[B]$	$F1[C]$	base		
Write AR			base + off	base + off	base + off			base + off	
Write DR						$a + F1[B]$	$F2[a+F1[B]] + F1[C]$	$F2[(a+F1[B]) + F1[C]]$	
Write status									
Write RF				a	b	c	sum	total	
Write Mem									total
Write PC	101	102	103	104	105	106	107	108	109

Giản đồ thời gian

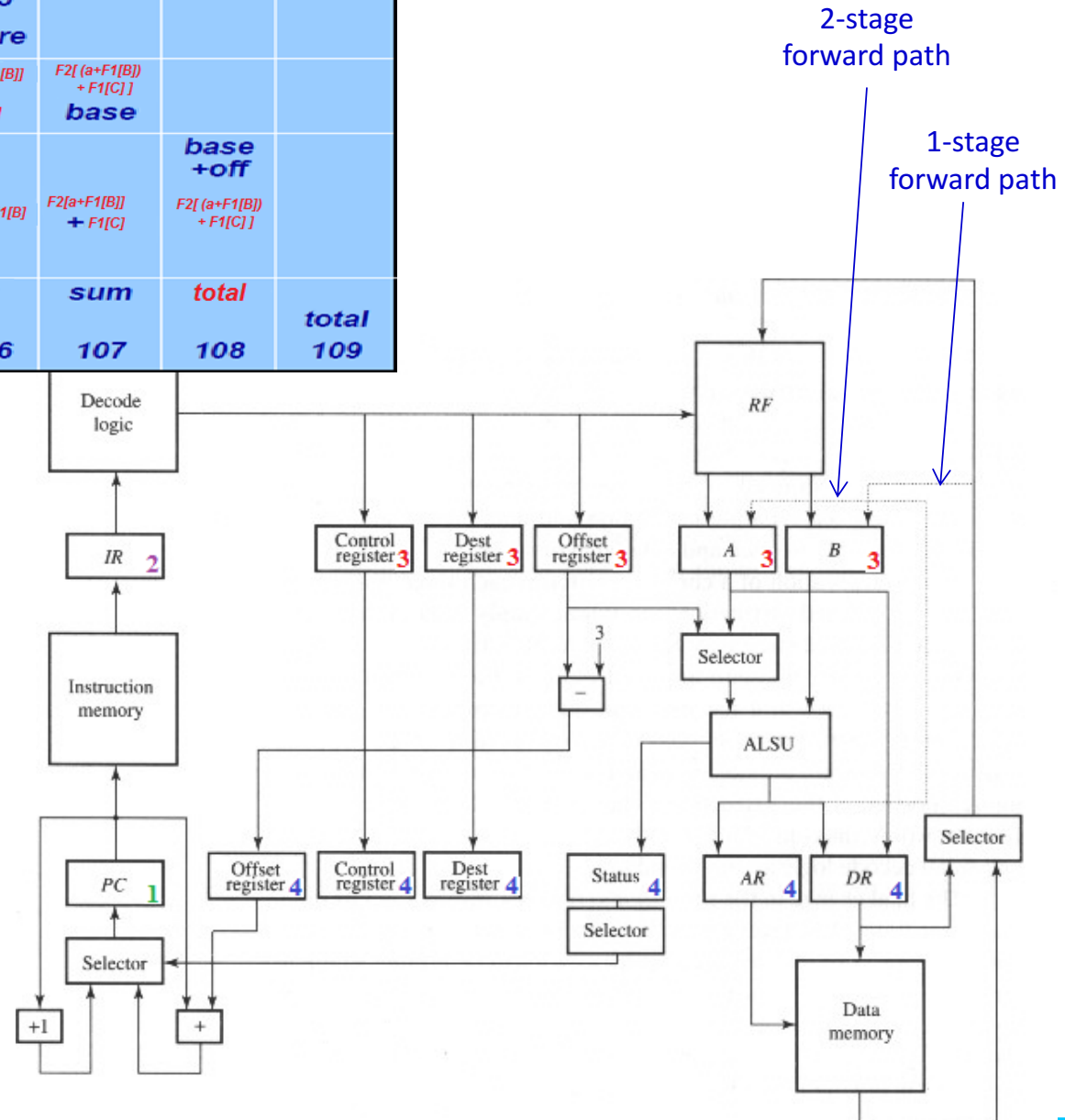


Hoạt động RISC dùng kỹ thuật nhìn trước

Clock cycle	0	1	2	3	4	5	6	7	8
Read PC	100	101	102	103	104	105			
Write IR	Load	Load	Load	Add	Add	Store			
Write A					a	$F2[a+F1[B]]$	$F2[(a+F1[B]) + F1[C]]$		
Write B		base	base	base	$F1[B]$	$F1[C]$	base		
Write AR			base + off	base + off	base + off			base + off	
Write DR						$a + F1[B]$	$F2[a+F1[B]] + F1[C]$	$F2[(a+F1[B]) + F1[C]]$	
Write status									
Write RF				a	b	c	sum	total	
Write Mem									total
Write PC	101	102	103	104	105	106	107	108	109

$\text{sum} = a + b$
 $\text{total} = \text{sum} + c$

Address	Instruction
100	Load a, base, off
101	Load b, base, off
102	Load c, base, off
103	Add sum, a, b
104	Add total, c, sum
105	Store total, base, off





Hoạt động RISC không dùng Dự đoán rẽ nhánh

```

If a ≥ b then
begin
  max = a
  min = b
end
else
begin
  max = b
  min = a
end
endif
    
```

Chương trình nguồn

Address	Instruction
100	Bgoeq a, b, +10
101	No-op
102	No-op
103	No-op
104	Move max, b
105	Move min, a
106	Jump + 6
107	No-op
108	No-op
109	No-op
110	Move max, a
111	Move min, b
112	

Chương trình hợp ngữ

Clock cycle	0	1	2	3	4	5	6	7	8	9	10
Read PC	100	101	102	103	104	105	106	107	108	109	112
Write IR	Bgoeq	No-op	No-op	No-op	Move	Move	Jump	No-op	No-op	No-op	
Write A		a				b	a				
Write B		b									
Write AR											
Write DR							b	a			
Write status			a < b								
Write RF								max	min		
Write Mem											
Write PC	101	102	103	104	105	106	107	108	109	112	

Giản đồ thời gian khi rẽ nhánh không được thực thi

Clock cycle	0	1	2	3	4	5	6	7	8	9	10
Read PC	100	101	102	103	110	111	112				
Write IR	Bgoeq	No-op	No-op	No-op	Move	Move					
Write A		a				a	b				
Write B		b									
Write AR											
Write DR							a	b			
Write status			a ≥ b								
Write RF								max	min		
Write Mem											
Write PC	101	102	103	110	111	112					

Giản đồ thời gian khi rẽ nhánh được thực thi



Hoạt động RISC có dùng Dự đoán rẽ nhánh

```

If a ≥ b then
begin
  max = a
  min = b
end
else
begin
  max = b
  min = a
end
endif
    
```

Chương trình nguồn

Address	Instruction
100	Bgoeq a, b, +4
101	Move max, b
102	Move min, a
103	Jump + 3
104	Move max, a
105	Move min, b
106	—

Chương trình hợp ngữ

Clock cycle	0	1	2	3	4	5	6	7	8
Read PC	100	101	102	103	104	105	106	106	
Write IR	Bgoeq	Move	Move	Jump	Move	Move			
Write A		a	b	a		a	b		
Write B		b							
Write AR									
Write DR				b	a		a		
Write status			a < b						
Write RF					max	min			
Write Mem									
Write PC	101	102	103	104	105	106	106		

result of
jump inst

Pipeline flush

Giản đồ thời gian khi rẽ nhánh ko được thực thi

Clock cycle	0	1	2	3	4	5	6	7	8
Read PC	100	101	102	103	104	105	106	106	
Write IR	Bgoeq	Move	Move	Jump	Move	Move			
Write A		a	b	a		a	b		
Write B		b							
Write AR									
Write DR				b			a	b	
Write status			a ≥ b						
Write RF								max	min
Write Mem									
Write PC	101	102	103	104	105	106			

result of
Bgoeq inst

Pipeline flush

Giản đồ thời gian khi rẽ nhánh được thực thi



Tóm tắt chương

❖ Các khái niệm đã giới thiệu:

- Tập lệnh
- Các loại lệnh
- Các chế độ đánh địa chỉ
- Chu kỳ thực thi lệnh
- Quy trình thiết kế bộ xử lý

❖ Bên cạnh đó trình bày:

- Thiết kế tập lệnh
- Lưu đồ tập lệnh
- Bố trí các khối thành phần
- Lưu đồ ASM
- Kiến trúc bộ xử lý

❖ Thiết kế bộ xử lý:

- Bộ xử lý CISC 16-bit
- Bộ xử lý RISC 32-bit (data-forwarding & branch-prediction)